Testing Strategy

Version: 1.0

Purpose

This document defines the testing strategy for Trading Platform Pro and its primary application, the Trading Cockpit.
The objective is to ensure:
• business correctness
• deterministic behaviour
• architecture stability
• trading workflow safety
• regression protection
• runtime reliability
• confidence in every code change
Trading-critical workflows require explicit scenario-based testing because technical side effects may affect external broker state.

Testing Philosophy

Testing is an integral part of implementation.
Every implementation shall be verifiable through automated tests where practical.
Testing is intended to:
• prevent regressions
• validate business behaviour
• preserve architecture contracts
• support refactoring
• validate failure behaviour
• protect trading-critical workflows
Tests shall verify observable behaviour and state.
Tests shall not merely reproduce implementation details.

Testing Principles

Tests shall be:
• deterministic
• readable
• isolated where appropriate
• explicit
• maintainable
• risk-oriented
Tests shall not depend on uncontrolled:
• wall-clock time
• network state
• LIVE broker services
• current market state
• developer-local configuration
• random data
Use controlled boundaries where required.

Risk-Based Testing

Testing depth depends on operational and business risk.
Higher-risk areas require stronger regression protection.

Trading-critical areas include:
• trading decisions
• order creation
• order validation
• order submission
• order cancellation
• broker acknowledgement
• execution processing
• position state
• reconciliation
• PAPER and LIVE environment selection
A small code change in a trading-critical workflow may require more testing than a large presentation-only change.

Test Model

The project uses multiple complementary test categories.
```text
Unit Tests
↓
Integration Tests
↓
Trading Workflow Regression Tests
↓
Runtime and System Tests
↓
Explicit PAPER Validation
```
These categories represent responsibility and risk.
They are not only execution size levels.
The majority of isolated business rules should remain covered by unit tests.
Trading-critical workflows require explicit scenario tests.

Unit Tests

Purpose:
Verify one business rule, state transition or isolated component behaviour.
Typical targets:
• Domain Entities
• Value Objects
• Domain Services
• Domain state transitions
• Application validation
• Application workflow decisions
• error translation logic
• configuration parsing
Unit tests shall be:
• fast
• deterministic
• focused
• independent from LIVE services

Domain Tests

Domain tests shall verify:
• invariants

• valid state transitions
• invalid state transitions
• Value Object validation
• financial value semantics
• unavailable data semantics
• Domain Exceptions
Example:
```text
Order CREATED
↓ acknowledge()
Order ACKNOWLEDGED
```
Tests shall also verify that invalid transitions are rejected.
Example:
```text
Order FILLED
↓ acknowledge()
Rejected
```
Domain tests shall not require:
• SQLAlchemy
• PySide6
• broker SDKs
• network services

Financial Value Tests

Financial values require explicit tests.
Tests shall verify where relevant:
• Decimal precision
• provider value normalization
• price validation
• quantity validation
• zero semantics
• unavailable value semantics
Missing financial data shall not be tested as zero unless zero is the actual business value.
Example distinction:
```text
None
→ unavailable
Decimal("0")
→ available value equal to zero
```

Application Tests

Application tests shall verify workflow coordination.
Typical targets:
• Commands
• Queries
• Application Services
• workflow state
• port interaction
• duplicate-prevention decisions
• reconciliation coordination
Application tests should use controlled port implementations.

Business logic shall not be mocked away.

Integration Tests

Purpose:
Verify collaboration across technical boundaries.
Typical targets:
• repository implementations
• SQLite persistence
• configuration loading
• dependency registration
• broker adapter translation
• market data adapter translation
• runtime service coordination
Integration tests shall validate actual boundary behaviour where practical.
Integration tests shall not require LIVE trading.

Repository Tests

Repository tests shall verify:
• save behaviour
• load behaviour
• Domain identity preservation
• persistence model translation
• transaction behaviour
• unavailable values
• state persistence
Repository tests shall not treat SQLAlchemy models as Domain Entities.
SQLite integration tests should use controlled temporary databases.

Configuration Tests

Configuration tests shall verify:
• default loading
• base configuration loading
• profile loading
• precedence
• environment variable overrides
• command-line overrides
• typed parsing
• validation
• invalid values
• PAPER and LIVE separation
Tests shall verify that LIVE is not activated through fallback.
Invalid critical LIVE configuration shall fail validation.

Broker Adapter Tests

Broker adapter tests shall verify provider translation boundaries.
Potential scenarios:
• connection state translation
• provider order translation
• provider identifier translation
• acknowledgement translation
• rejection translation

• execution translation
• partial fill translation
• provider error translation
Broker adapter tests shall not test trading decisions.
Provider-specific models shall remain inside Infrastructure test boundaries.

Market Data Adapter Tests

Market data adapter tests shall verify:
• quote translation
• source timestamp preservation
• unavailable values
• stale data state
• connection state translation
• subscription activation
• subscription failure
• subscription closure
Tests shall preserve the distinction between:
```text
Current
Stale
Unavailable
Disconnected
```
Cached data shall not be tested as current provider data unless freshness state explicitly supports that result.

Trading Workflow Regression Tests

Trading workflow regression tests protect business-critical multi-step behaviour.
These tests are mandatory for implemented trading-critical workflows.
Potential targets include:
• Trading Candidate lifecycle
• Trading Decision lifecycle
• Order lifecycle
• Duplicate submission prevention
• Broker acknowledgement
• Broker rejection
• Partial fills
• Fills
• Position lifecycle
• Reconciliation
Trading workflow tests shall verify complete state sequences.

Order Lifecycle Regression Tests

Order lifecycle tests shall preserve the distinction between:
```text
Submission Requested
↓
Validated
↓
Transmitted
↓
Acknowledged or Rejected
↓
```

Partially Filled or Filled
```

Tests shall not use one generic `submitted` state for multiple lifecycle stages.
A successful adapter call shall not be treated as broker acknowledgement unless broker-derived acknowledgement state exists.
---

# Order Submission Tests

Order submission tests shall verify where implemented:
• valid submission request
• invalid submission request
• local validation failure
• transmission
• broker acknowledgement
• broker rejection
• unknown acknowledgement state
• timeout
• disconnect
Tests shall verify persisted state where order state is persisted.
Logs alone shall not be treated as authoritative business state.
---

# Duplicate Submission Tests

Duplicate submission risk requires explicit regression tests.
Tests shall verify where implemented:
• stable order identity
• stable submission identity
• duplicate detection
• duplicate blocking
• repeated application callback
• repeated broker message
• timeout followed by repeated workflow invocation
• disconnect followed by reconnect
• application restart or recovery
Unknown external order state shall not automatically result in repeated submission.
CI shall fail when required duplicate-submission regression tests fail.
---

# Retry Tests

Retry behaviour shall be tested explicitly where implemented.
Tests shall verify:
• retry eligibility
• attempt count
• previous failure state
• retry reason
• resulting state
Automatic retry may be tested for approved read operations.
Automatic order submission retry shall not exist unless explicitly defined and approved by the owning order workflow.
Generic retry behaviour shall not hide duplicate execution risk.
---

# Broker Acknowledgement Tests

Broker acknowledgement semantics require explicit tests.
Tests shall verify the difference between:
```

```text
Adapter call returned
```
and:
```text
Broker acknowledgement received
```
A local successful method return shall not automatically transition the order to `ACKNOWLEDGED`.
Repeated acknowledgement messages shall be handled deterministically.

Broker Rejection Tests

Broker rejection tests shall verify:
• normalized rejection state
• provider error translation
• order state
• persisted state where applicable
• operational event emission where applicable
A broker rejection is not automatically a runtime failure.
The application shall remain in a valid deterministic state.

Partial Fill Tests

Partial fill tests shall verify:
• filled quantity
• remaining quantity
• order state
• repeated execution messages
• position impact where implemented
Repeated provider messages shall not duplicate filled quantity.
Partial fill state shall remain distinct from filled state.

Fill Tests

Fill tests shall verify:
• final filled quantity
• execution identity
• order state
• position impact
• persisted state where applicable
Repeated fill messages shall be handled deterministically.
A fill shall not be inferred solely from UI state.

Order Cancellation Tests

Order cancellation tests shall verify where implemented:
• cancellation request
• cancel-pending state
• broker cancellation acknowledgement
• cancelled state
• cancellation rejection
• execution during cancellation
A local cancellation request shall not automatically mean the broker order is cancelled.

Disconnect Tests

Broker disconnect tests shall verify:
• connection state
• runtime capability state
• active workflow state
• unknown external order state
Disconnect shall not automatically trigger order resubmission.
Tests shall verify that connection recovery and order submission remain separate concerns.

Reconnect Tests

Reconnect tests shall verify:
• reconnect state
• connection recovery
• capability state recovery
• repeated connection events
Reconnect shall not blindly repeat:
• order submission
• order cancellation
• trading decisions
Any recovery workflow affecting order state requires explicit tests.

Position Lifecycle Tests

Position tests shall verify where implemented:
• position opening
• position update
• partial quantity changes
• position closing
• position closed state
• execution-derived changes
Position state shall not be inferred solely from presentation state.
Financial values shall preserve explicit precision and availability semantics.

Reconciliation Tests

Reconciliation workflows require explicit regression tests.
Tests shall verify where implemented:
• local state loading
• broker state loading
• no discrepancy
• discrepancy detection
• discrepancy classification
• action-required state
• reconciliation failure
Potential discrepancy scenarios include:
• missing local order
• missing broker order
• order state mismatch
• position state mismatch
• quantity mismatch
Reconciliation tests shall verify observation and classification separately from repair actions.

Reconciliation Repair Tests

If reconciliation repair workflows are introduced, they require separate explicit tests.
Tests shall verify:
• repair authorization
• repair preconditions
• affected business state
• persisted state
• failure behaviour
• repeated repair invocation
Monitoring and logging shall never be tested as reconciliation repair owners.
Repair behaviour belongs to explicit Application workflows.

Runtime Tests

Runtime tests shall verify technical lifecycle behaviour.
Potential scenarios:
• deterministic startup order
• startup failure
• runtime state transitions
• degraded state
• task ownership
• task failure
• cancellation
• shutdown
• repeated shutdown request
Runtime tests shall use controlled technical boundaries.

Broker Lifecycle Runtime Tests

Runtime tests may verify:
```text
Initialized
↓
Connecting
↓
Connected
↓
Operational
```
Failure scenarios may include:
• Disconnected
• Reconnecting
• Failed
Broker capability state shall remain distinguishable from overall runtime state.

Market Data Lifecycle Runtime Tests

Runtime tests may verify:
```text
Initialized
↓
Connecting
↓
Connected
```

↓
Subscription Ready
```
Failure scenarios may include:
• Disconnected
• Degraded
• Subscription Failed
Market data state shall remain distinguishable from broker state.
---
# Subscription Lifecycle Tests

Subscription tests shall verify:
• requested
• active
• stale
• failed
• closing
• closed
Tests shall verify ownership and cleanup.
Shutdown tests shall verify active subscriptions are closed before market data infrastructure is disposed.
---
# Background Task Tests

Background task tests shall verify:
• registration
• task identity
• startup
• running state
• failure observation
• cancellation
• shutdown
Long-running application tasks shall not be tested as unmanaged fire-and-forget tasks.
Task ownership is part of runtime correctness.
---
# AsyncIO Tests

AsyncIO tests shall verify where relevant:
• cancellation
• timeout
• task failure
• task cleanup
• event loop responsiveness
Avoid arbitrary sleeps.
Prefer deterministic synchronization primitives and controlled events.
Example:
```python
event = asyncio.Event()
```
Tests shall not depend on timing luck.

Timeout Tests

Timeout tests shall verify the resulting state explicitly.
A timeout shall remain distinguishable from:
```

• rejection
• cancellation
• disconnect
• validation failure
Do not assert only that an exception occurred.
Assert the resulting workflow state and side effects.

Cancellation Tests

Cancellation tests shall verify:
• cancellation propagation
• resource release
• resulting state
• absence of false external acknowledgement
Cancellation shall not be tested as successful external completion.
Business-critical cancellation behaviour requires explicit state assertions.

Shutdown Tests

Controlled shutdown tests shall verify the documented shutdown sequence where implemented.
Potential checks include:
1. New trading actions rejected.
2. New workflow intake stopped.
3. New subscription requests stopped.
4. Background tasks cancelled or completed according to policy.
5. Active subscriptions closed.
6. Market data lifecycle stopped.
7. Broker lifecycle stopped.
8. State persisted where required.
9. Runtime entered `Stopped`.
Shutdown shall not blindly cancel broker orders unless an explicit Application policy owns that action.

System Tests

System tests validate complete application workflows across multiple capabilities.
Potential scenarios:
• application startup
• runtime initialization
• configuration loading
• persistence initialization
• broker lifecycle
• market data lifecycle
• workspace restoration
• controlled shutdown
System tests shall not require LIVE trading.
System tests may use controlled fake providers.

Presentation Tests

Presentation tests should focus on UI behaviour and Application interaction.
Potential targets:
• widget initialization
• workspace restoration
• command invocation

• operational state presentation
• PAPER and LIVE visibility
Presentation tests shall not reproduce Domain business rules.
Widgets shall not be tested as direct broker clients.

PAPER and LIVE Safety Tests

PAPER and LIVE separation requires automated regression tests.
Tests shall verify:
• PAPER profile selects PAPER
• LIVE profile selects LIVE only explicitly
• invalid LIVE configuration fails
• no fallback activates LIVE
• PAPER does not silently use LIVE broker configuration
• environment context reaches Application workflows
• environment state is visible to Presentation where required
Automated tests shall not execute LIVE orders.

Test Doubles

Use test doubles to control architecture boundaries.
Potential test doubles include:
• fake repositories
• fake broker ports
• fake market data ports
• fake clocks
• fake event dispatchers
• controlled runtime services
The selected test double shall support the test purpose.

Fakes

Prefer fakes when realistic deterministic behaviour is useful.
Example:
```python
class FakeBrokerOrderPort:
...
```
A fake may model:
• acknowledgement
• rejection
• timeout
• disconnect
• repeated provider message
Fakes shall not contain production business logic.

Mocks

Mocks may verify important boundary interactions.
Use mocks carefully.
Good use cases:
• verify one submission request
• verify no duplicate submission
• verify cancellation was not requested
• verify a port was not called after validation failure

Avoid excessive mock call chains that reproduce implementation structure.

Stubs

Stubs may provide controlled responses.
Examples:
• fixed market quote
• fixed broker connection state
• fixed configuration value
Stubs shall make test scenarios explicit.

Clock Control

Time-dependent tests shall use a controlled clock where practical.
Avoid direct dependence on:
```python
datetime.now()
```
inside business-critical tests.
A controlled clock supports deterministic testing of:
• timestamps
• timeouts
• scheduled behaviour
• reconciliation windows

Mocking Business Logic

Do not mock the business rule being tested.
Example:
If testing duplicate submission prevention, do not mock:
```text
DuplicateSubmissionPolicy
```
to return the expected result.
Exercise the actual rule.
Mock or fake the external broker boundary instead.

Test Organization

Initial test structure may include:
```text
tests/
■■■ unit/
■■■ integration/
■■■ regression/
■■■ system/
■■■ performance/
```
The structure may evolve with implemented capabilities.
Tests should mirror production capabilities where practical.
Do not create empty test directories for hypothetical future capabilities.

Test Naming

Test files:

```text
test_.py
```
Examples:
```text
test_order.py
test_order_submission.py
test_duplicate_submission.py
test_reconciliation.py
test_runtime.py
```
Test names shall describe behaviour.
Prefer:
```python
def test_rejects_duplicate_submission_for_same_submission_key() -> None:
...
```
Avoid:
```python
def test_order_1() -> None:
...
```

Test Structure
Tests should clearly communicate:
• given state
• operation
• expected result
Example conceptual structure:
```text
Given
Order is transmitted
When
Broker acknowledgement is received
Then
Order becomes acknowledged
```
Explicit helper functions may be used where they improve readability.
Avoid large generic test builders that hide scenario meaning.

Assertions
Assertions shall verify meaningful behaviour.
Prefer:
```python
assert order.status is OrderStatus.ACKNOWLEDGED
```
Avoid weak assertions such as:
```python
assert result is not None
```
when exact state is known.
Trading-critical tests shall assert:
• resulting state
• external interaction count where relevant

• persisted state where relevant

Regression Tests

Every bug fix should include a regression test whenever practical.
The preferred regression workflow is:
1. Reproduce the previous failure.
2. Verify the test fails for the previous behaviour.
3. Implement the correction.
4. Verify the regression test passes.
5. Run affected broader tests.
Trading-critical bug fixes require regression coverage unless a concrete technical reason prevents automation.

Flaky Tests

Flaky tests are defects.
Do not:
• ignore flaky tests indefinitely
• add arbitrary sleeps
• rerun until green as the normal solution
• weaken assertions without understanding the failure
A flaky test shall be:
1. identified
2. investigated
3. stabilized
4. documented if temporary quarantine is unavoidable
Trading-critical flaky tests require priority investigation.

Test Data

Test data shall be:
• deterministic
• minimal
• scenario-specific
• readable
Avoid production account data.
Avoid real credentials.
Avoid uncontrolled provider payload dumps.
Provider payload fixtures shall be sanitized.

Test Isolation

Tests shall not depend on execution order.
Each test shall own or reset its state.
Temporary persistence shall be isolated.
Global mutable test state shall be avoided.
A test passing only after another test has executed is invalid.

Continuous Integration

Every relevant repository change shall execute automated quality gates.
CI test categories may include:
• Unit Tests
• Integration Tests

• Trading Workflow Regression Tests
• Runtime or System Tests
Failed required tests block integration.
CI shall not require LIVE trading.

Local Development

Start with the smallest relevant test scope.
Example:
```bash
pytest tests/unit/test_order.py -q
```
Then run the affected test category.
Example:
```bash
pytest tests/unit -q
```
For trading-critical changes run affected regression tests.
Example:
```bash
pytest tests/regression -q
```
Before merge, run the required project quality gates.

Test Coverage

Every new capability requires appropriate test coverage.
Preferred priorities:
1. business invariants
2. trading-critical workflows
3. state transitions
4. external side-effect coordination
5. error handling
6. regression scenarios
7. architecture boundaries
Coverage is a quality indicator.
Coverage percentage is not a safety guarantee.
High coverage does not prove duplicate submission safety or correct broker acknowledgement semantics.

Performance Testing

Performance tests should verify measurable operational concerns.
Potential targets:
• startup duration
• UI responsiveness
• AsyncIO event loop lag
• market data update paths
• structured logging overhead
• memory consumption
Performance tests shall run separately from normal unit tests where appropriate.
Optimize only after measurement.
Performance improvements shall not weaken business correctness.

PAPER Validation

Trading-related implementations should be validated in PAPER mode before operational LIVE use where the workflow interacts with a real broker boundary.
PAPER validation may include:
• broker connection
• market data connection
• order submission workflow
• broker acknowledgement
• rejection handling
• execution handling
• cancellation handling
• logging
• reconciliation
PAPER validation is not a replacement for deterministic automated tests.
PAPER behaviour may depend on broker and market conditions.

PAPER Validation Procedure

A PAPER validation shall define:
1. Validation objective.
2. Expected workflow.
3. Expected broker state.
4. Expected local state.
5. Expected logs.
6. Failure condition.
7. Cleanup or reconciliation requirement.
Results should be recorded for business-critical workflow validation.
Do not treat an unstructured manual test as complete regression coverage.

LIVE Validation

LIVE validation requires explicit approval.
LIVE validation shall never be part of normal CI.
Before LIVE validation verify:
• automated tests pass
• trading regression tests pass
• PAPER validation completed where required
• LIVE environment is explicit
• order parameters are reviewed
• expected external side effect is understood
• reconciliation procedure is available
LIVE validation shall use the smallest operationally meaningful scope.
Do not use LIVE trading to discover basic implementation defects that deterministic or PAPER tests should detect.

Documentation Testing

Markdown under:
```text
docs/
```
is the documentation source of truth.
When documentation changes, validate generation using:
```bash
python scripts/generate_docs.py
```
The script shall complete successfully.

Generated DOCX and PDF files shall not be re-read for content validation.
Documentation tests validate source compatibility and generation success.

Architecture Tests

Architecture boundaries should be tested automatically where practical.
Potential rules:
• Domain does not depend on Infrastructure
• Domain does not depend on Presentation
• Application does not depend on Presentation
• Presentation does not access concrete broker adapters directly
• SQLAlchemy models do not leak into Domain
• PySide6 types do not leak into Domain
• provider models do not leak into Domain
Architecture tests shall validate documented boundaries.
Do not create architecture tests for hypothetical boundaries.

Test Review

Before merging verify:
• tests are deterministic
• tests are readable
• tests verify behaviour
• assertions are meaningful
• business logic is not mocked away
• test doubles represent controlled boundaries
• no LIVE credentials are required
• no production account data is used
• no arbitrary sleeps hide timing problems
• no duplicated tests exist without additional value
• trading-critical scenarios have regression coverage
• PAPER and LIVE separation is tested
• documentation generation passes where required

Testing Review Checklist

Before completing a change verify:
• risk level identified
• affected capability identified
• unit test impact evaluated
• integration test impact evaluated
• trading regression impact evaluated
• runtime or system test impact evaluated
• state transitions tested
• unavailable financial data tested where relevant
• timeout tested where relevant
• cancellation tested where relevant
• broker acknowledgement semantics tested where relevant
• duplicate submission risk tested where relevant
• retry behaviour tested where relevant
• disconnect behaviour tested where relevant
• reconnect behaviour tested where relevant
• partial fill tested where relevant
• repeated provider messages tested where relevant
• persistence state tested where relevant
• reconciliation tested where relevant

• PAPER and LIVE separation tested where relevant
• deterministic boundaries used
• business logic not mocked away
• focused tests passed
• required regression tests passed
• documentation generation passed where required

Related Documents

• Product_Vision.md
• Product_Roadmap.md
• Project_Overview.md
• Architecture.md
• Domain_Model.md
• Infrastructure.md
• Technical_Specifications.md
• API_Guidelines.md
• Coding_Standards.md
• Development_Guidelines.md
• Git_Workflow.md
• Configuration.md
• Runtime.md
• Logging.md
• Monitoring.md
• CI_CD.md
• AGENTS.md